SEMANA 3 · AULA 2 · 04/11

MÚLTIPLAS SKILLS

Skills finas de code review: do monólito à suíte com agente revisor

Felipe Oliveira · IA Aplicada à Produtividade no Desenvolvimento

<LAB365> SENAI

ONDE PARAMOS

?

achados confirmados ontem (melhor equipe)

?

falsos positivos somados

12

requisitos na spec

72%

meta de hoje (Rajan 2025)

A PERGUNTA DE HOJE

Ontem a v0 revisou o vizinho. Com dez skills instaladas, qual dispara? E como três revisores viram um relatório só?

PARTE 1: GATILHO

Descrições competem pelo mesmo 1% da janela. Near-miss é o teste que vale.

PARTE 2: COMPOSIÇÃO

Granularidade, orquestração, subagentes em paralelo, hook e steering no lugar certo.

AGENDA DA AULA

19h00	Placar da revisão cruzada e a pergunta de hoje	ABERTURA
19h10	Bloco 3: o problema do gatilho	TEORIA
19h30	Atividade 2.1: O gatilho certo	PRÁTICA
20h00	Intervalo	PAUSA
20h10	Bloco 4: granularidade e composição	TEORIA
20h30	Atividade 2.2: suíte + agente revisor	PRÁTICA
21h15	Disputa 2: revisão cruzada, rodada 2	DISPUTA

ENTREGÁVEL DE HOJE

Sua suíte v1

- 3 skills review-* especializadas
- 1 review-orchestrator
- agente revisor (json ou .md)
- 1 hook de testes ao salvar
- reports/<equipe>-v1.md no PR da vizinha

BLOCO 3: O PROBLEMA DO GATILHO

Uma descrição boa não basta quando há dez descrições

DEZ DESCRIÇÕES, UM GATILHO

LISTAGEM NO CONTEXTO (NÍVEL 1)

review-security	description: ...
review-correctness	description: ...
review-performance	description: ...
pr-summary	description: ...
pytest-runner	description: ...
git-commit-msg	description: ...
lint-fix	description: ...
docs-writer	description: ...



"olha esse PR, acho que tem algo errado no estorno"

o modelo escolhe pela proximidade semântica

1%
da janela para a listagem

1.536
caracteres por descrição

FALHAS TÍPICAS DO GATILHO

GÊMEAS

Duas descrições parecidas

EFEITO

As duas disparam (instruções conflitantes) ou nenhuma. Diferencie pelo que cada uma NÃO faz.

ESTREITA

Só dispara com a palavra exata

EFEITO

"revisa a segurança" dispara; "tem alguma falha nisso?" não. Liste os jeitos de pedir.

LARGA

Dispara em tudo que tem código

EFEITO

"escreve testes" carrega a skill de review. Diga o que ela não cobre.

TAREFA SIMPLES

O agente acha que não precisa

EFEITO

Pedido de um passo não consulta skill. Use /nome ou mencione o fluxo na descrição.

NEAR-MISS: O TESTE QUE VALE

NEGATIVO FRACO

Não testa nada

exemplo	"escreve uma função fibonacci"
exemplo	"qual a previsão do tempo?"
por quê	sem vocabulário em comum; qualquer descrição passa

vs

NEGATIVO FORTE

Compartilha palavras, pede outra coisa

exemplo	"adiciona log de auditoria em refund()" (puxa security)
exemplo	"cria um índice pra customer_id" (puxa performance)
por quê	só uma descrição precisa dispara aqui

Receita: 8 a 10 prompts que devem disparar, 8 a 10 near-misses, 3 rodadas cada; taxa > 0,5 nos positivos e < 0,5 nos negativos.

COMO MEDIR O GATILHO

PASSO 1

20 prompts

8 a 10 positivos variando formalidade e explicitude; 8 a 10 near-misses.

PASSO 2

Treino 60/40

60% para guiar as mudanças, 40% só para conferir se generalizou.

PASSO 3

3 rodadas

Modelo é não determinístico: taxa de disparo, não acerto único.

PASSO 4

Revisar

Falhou positivo: amplie o escopo. Disparou negativo: diga o que não faz.

PASSO 5

Escolher

A melhor iteração pela validação, não a última. Até 5 iterações.

Armadilha: colar as palavras do prompt que falhou na descrição é sobreajuste. Descreva a categoria, não a frase.

ATIVIDADE 2.1: O GATILHO CERTO

Quatro descrições, vinte prompts, cinco rodados de verdade

O GATILHO CERTO: COMO JOGAR

PASSO 1

Escrever (10 min)

Descrições de review-security, review-correctness, review-performance e pr-summary.

PASSO 2

Prever (10 min)

20 prompts projetados. Para cada um, qual skill dispara? Ou nenhuma?

PASSO 3

Rodar (10 min)

5 prompts sorteados rodam na sua ferramenta com as suas descrições.

EVENTO	PONTOS
previsão correta	+1
disparo real correto	+2
disparo em near-miss (11 a 20)	-1

DICA

Diga o que não faz

"Não escreve testes nem faz deploy."

BLOCO 4: COMPOSIÇÃO

Granularidade, orquestração, subagentes, hook e steering no lugar certo

SKILL É FUNÇÃO, NÃO MONÓLITO

ESTREITA DEMAIS

review-sqli

problema 10 skills para uma tarefa

efeito sobrecarga e instruções conflitantes

COERENTE

review-security

cobre injeção, auth, segredos, logs

efeito dispara com precisão e compõe bem

LARGA DEMAIS

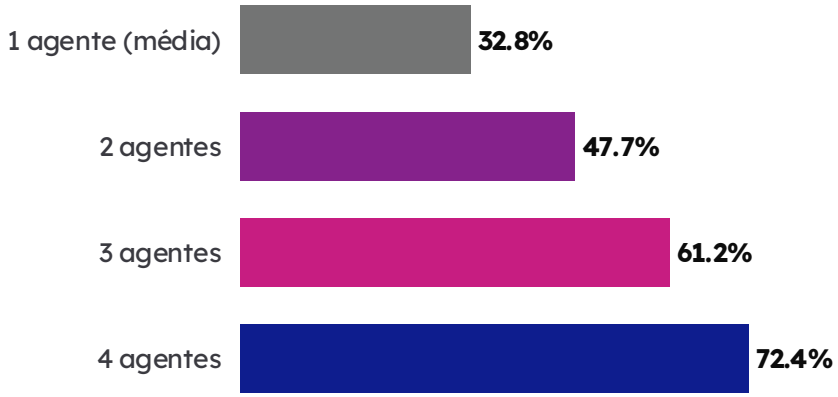
review-all

problema 930 linhas, 40 regras

efeito não dispara com precisão; fábrica de FP

> **"Deciding what a skill should cover is like deciding what a function should do."** agentskills.io, Best practices.

REVISORES ESPECIALIZADOS SOMAM



acurácia na detecção; ganhos de +14,9, +13,5 e +11,2 pontos

72,4%

4 revisores em paralelo

32,8%

um revisor só

0,05 a 0,25

correlação entre os revisores

79,3%

melhor par: correção e desempenho

Decisão de projeto: 3 ou 4 revisores de categorias pouco correlacionadas + 1 consolidador. O quinto revisor custa créditos e rende pouco.

CINCO FORMAS DE COMPOR

A) EMPILHAR

/review-sec /review-perf

ONDE

Claude Code e Copilot: skills como /comando na mesma mensagem.

B) ORQUESTRADORA

Skill que referencia skills

ONDE

Qualquer agente: "para segurança, siga review-security". Tudo na mesma janela.

C) AGENTE CUSTOM

revisor.json / revisor.md

ONDE

Nas três: prompt próprio, tools limitadas, skills pré-carregadas.

D) SUBAGENTES

Um por skill, em paralelo

ONDE

Nas três. Contexto isolado; só o resumo volta ao principal.

E) HOOK

Fora do modelo

ONDE

PostFileSave em .py roda pytest. Regra que deve valer toda vez não fica na skill.

+ STEERING

Convenções sempre presentes

ONDE

O arquivo de ontem, com poucas linhas. O que o revisor nunca pode esquecer.

CADA COISA NO MECANISMO CERTO



SEMPRE

`steering (AGENTS.md)`

Sempre carregado. Centavos, sqlite com ?, UTC, PAN mascarado. 8 linhas.



SOB DEMANDA

`skills/review-*/SKILL.md`

Dispara pela descrição ou pelo agente revisor. Método, CWEs, formato do achado.



POR EVENTO

`hook de testes`

Ao salvar .py (ou ao terminar), roda pytest -q e injeta o resultado. Não depende do modelo.



Por que não tudo na skill: o modelo pode parar de seguir a skill depois de uma compactação de contexto. O que precisa valer toda vez vai para hook.

SUBAGENTES: O QUE ISOLAM

COMPARTILHADO COM O PRINCIPAL

O que o subagente herda

steering	as mesmas convenções
MCP	os mesmos servidores
arquivos	o mesmo workspace
permissões	regra do mais estrito

ISOLADO POR SUBAGENTE

O que ele tem só para si

histórico	conversa própria
janela	contexto próprio, não polui o pai
retorno	só o resumo final volta
tools	lista própria, mais estreita

Permissões: o subagente nunca pode mais que o pai. Kiro: regra do mais estrito; Claude Code e Copilot: tools como allowlist.

O AGENTE REVISOR (EXEMPLO KIRO)

.kiro/agents/revisor.json

```
{
  "name": "revisor",
  "description": "Orquestra a revisão com revisores especializados",
  "prompt": "Siga review-orchestrator: delegue em paralelo,
             consolide sem duplicatas, descarte sem evidência.",
  "tools": ["read", "shell",
            "subagent/review-security",
            "subagent/review-correctness",
            "subagent/review-performance"],
  "resources": ["skill://.kiro/skills/review-orchestrator/SKILL.md",
                "file://.kiro/steering/conventions.md"],
  "permissions": { "rules": [
    { "capability": "subagent", "match": ["review-*"], "effect": "allow" } ] }
}
```

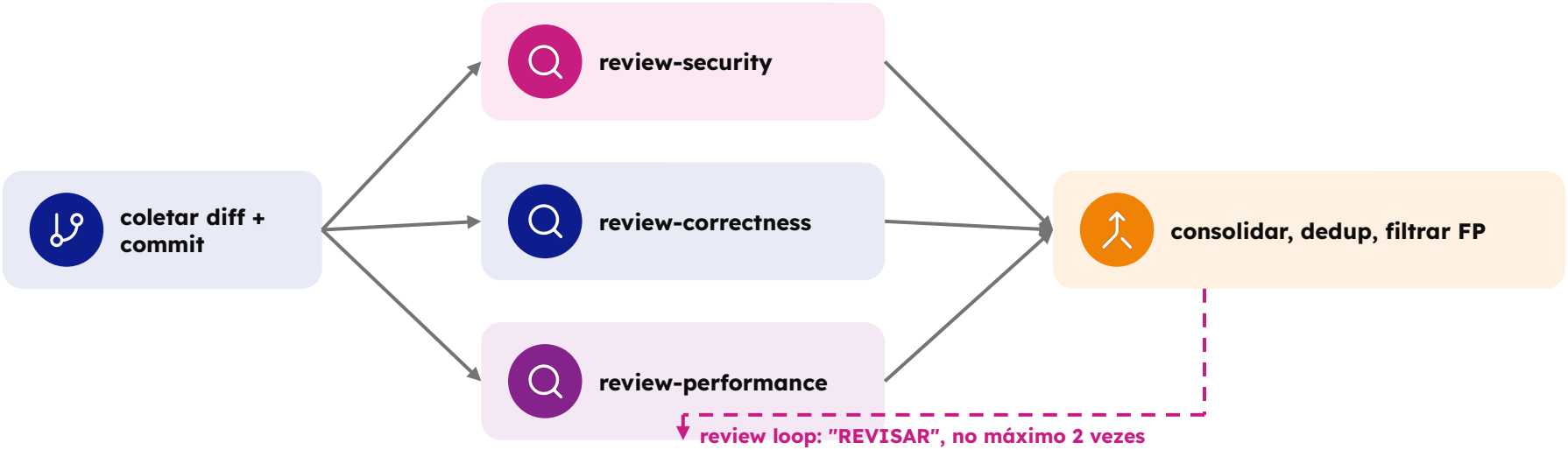
- **tools:** subagent/<nome>, nunca * nem @builtin
- **resources:** skill:// e file:// carregam ao iniciar
- **permissions:** allow para review-* evita o prompt
- **subagentes:** um .json por revisor, cada um com sua skill
- **uso:** "use o agente revisor no repositório"

O MESMO AGENTE EM TRÊS FERRAMENTAS

	KIRO	CLAUDE CODE	VS CODE + COPILOT
arquivo	.kiro/agents/revisor.json	.claude/agents/revisor.md	.github/agents/revisor.agent.md
prompt	campo prompt	corpo do markdown	corpo do markdown
tools	tools: [read, shell, subagent/...]	tools: Read, Grep, Bash	tools: ['read', 'search', ...]
skills	resources: skill://.../SKILL.md	skills: [review-orchestrator]	lê .github/skills/ pela descrição
subagentes	tools: subagent/review-*	delega por nome ou description	handoffs: ou chamada como subagente
isolamento	janela própria; só o resumo volta	janela própria; só o resumo volta	janela própria; só o resumo volta

O que não muda: a skill. Os quatro SKILL.md da suíte são os mesmos nas três ferramentas; só o arquivo do agente troca de formato.

DAG, PARALELO E REVIEW LOOP



Custo: 3 janelas em paralelo gastam 3 vezes os tokens de entrada. O ganho de recall precisa pagar isso; a Disputa 2 mede.

O HOOK QUE RODA OS TESTES

.kiro/hooks/tests-on-save.json

```
{
  "version": "v1",
  "hooks": [{
    "name": "pytest-on-save",
    "trigger": "PostFileSave",
    "matcher": "\\.py$",
    "action": { "type": "command",
      "command": "python -m pytest -q --tb=line | tail -n 15" },
    "timeout": 60
  }]
}
```

FERRAMENTA	ONDE
Kiro	.kiro/hooks/*.json
Claude Code	settings.json: hooks
Copilot	.github/hooks/*.json
eventos	save, pre/post tool, stop

Dois tipos de ação: command (shell, determinístico) e agent (injeta um prompt). Lint, testes e validadores vão em command.

MAIS ACHADOS NÃO É MELHOR

328

de 433 alarmes de SAST eram falsos (Tencent)

10 a 20 min

de dev por alarme inspecionado

94 a 98%

dos FP removidos por LLM + estática

US$ 0,12

custo máximo por alarme filtrado



O CONSOLIDADOR

review-orchestrator

- mesmo arquivo e linha (±3): mantém um
- evidência cita trecho real da linha? senão, descarta
- achado fora da área do revisor só com evidência
- seção "Descartados" com o motivo
- validate_report.py antes de entregar

ATIVIDADE 2.2 E DISPUTA 2

Do monólito à suíte: três revisores, um orquestrador, um agente

SUÍTE V1: PASSO A PASSO

PASSO 1

Quebrar

code-review-<grupo> vira review-security, review-correctness, review-performance.

PASSO 2

Enriquecer

security carrega cwe-top25.md e pede diff + commit via collect_diff.sh.

PASSO 3

Orquestrar

review-orchestrator: delega, deduplica, descarta sem evidência.

PASSO 4

Agentizar

revisor.json + 3 subagentes + hook tests-on-save.json.

PASSO 5

Rodar

No MESMO projeto vizinho de ontem: "use o agente revisor contra a spec".

Limite: no máximo 4 skills por suíte. Rajan mostra ganho decrescente; o resto é crédito gasto.

ARQUITETURA ALVO DA SUÍTE

iapd-26t3-eNN-pagamentos/.kiro/ (exemplo Kiro)

```
.kiro/
├── skills/
│   ├── review-security/
│   │   ├── SKILL.md
│   │   └── references/cwe-top25.md
│   ├── review-correctness/SKILL.md
│   ├── review-performance/SKILL.md
│   └── review-orchestrator/SKILL.md
├── agents/
│   ├── revisor.json
│   ├── review-security.json
│   ├── review-correctness.json
│   └── review-performance.json
├── steering/conventions.md
└── hooks/tests-on-save.json
```

4 SKILLS, IGUAIS NAS TRÊS

3 especializadas + 1 orquestradora. Só a pasta raiz muda: .kiro, .claude ou .github.

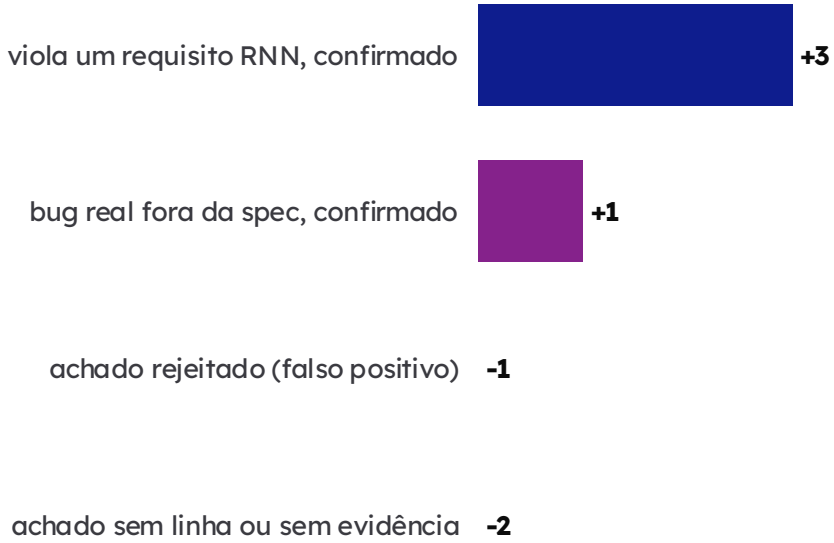
4 AGENTES, FORMATO DA FERRAMENTA

1 revisor que delega + 3 subagentes: .json no Kiro, .md no Claude Code, .agent.md no Copilot.

1 STEERING + 1 HOOK

Convenções sempre presentes; pytest a cada save ou ao terminar.

DISPUTA 2: REVISÃO CRUZADA, RODADA 2



BÔNUS DE EVOLUÇÃO

v0 para v1

- +5 maior ganho de achados confirmados v0 para v1
- +3 maior redução de falsos positivos
- mesmo projeto vizinho de ontem
- anote os créditos gastos
- quantos revisores valeram a pena?

O QUE FICA E O QUE VEM

FICA

Descrições competem: teste com near-miss. Revisores pouco correlacionados somam; o consolidador filtra.

TRAGA A SUÍTE FUNCIONANDO

Quinta ela é a arma do Blue Team no Torneio, e a base do seu trabalho final.

QUINTA (05/11)

Spec, steering e skills do SEU trabalho final; skills seguras; o Torneio Red x Blue.

LISTA DA SEMANA

Suíte no GitHub, evals.json + benchmark, auditoria de uma skill pública, 5 linhas do projeto. Entrega 09/11.

4

skills por suíte, no máximo

3

subagentes em paralelo

50

min de Torneio na quinta

5

bugs que você vai plantar


<LAB365>
SENAI